SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4) Assessment Tool Weightage: 40%

QUESTIONS:5 **Total Marks:25**

ANNEXURE A
APPLICATION BASED QUESTIONS

Code of Conduct:

I N. Tharun Sai / 192472382 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

N. Tharun Sai
Signature of the student:

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.
5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

Question 1: Smart City Surveillance System

Introduction

A smart city surveillance system is designed to continuously monitor public spaces using multiple cameras connected through a network. These systems generate massive amounts of video data that must be efficiently transferred between input/output (I/O) devices, memory, and the CPU for real-time analysis and threat detection. The overall performance of the surveillance system depends on how effectively these hardware components communicate and process data. During peak traffic periods, simultaneous access to system resources can create bottlenecks, leading to delays in video processing and slower response times. Understanding the interaction between the CPU, memory, and I/O devices, along with the impact of different bus architectures and instruction execution techniques, is essential for improving system performance and ensuring timely threat detection.

Applications

- Smart traffic monitoring
- Railway station surveillance
- Airport security
- Bank security systems
- Smart parking
- Public safety monitoring

Sample Python Program

```
import cv2

cap = cv2.VideoCapture("video.mp4")

while True:
    ret, frame = cap.read()
    if not ret:
        break
```

```
    cv2.imshow("Camera", frame)

    if cv2.waitKey(1) == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

- **Bus Structures**

Single Bus Architecture

```
CPU
 |
----------------------
|      |        |
RAM    HDD     Camera
```

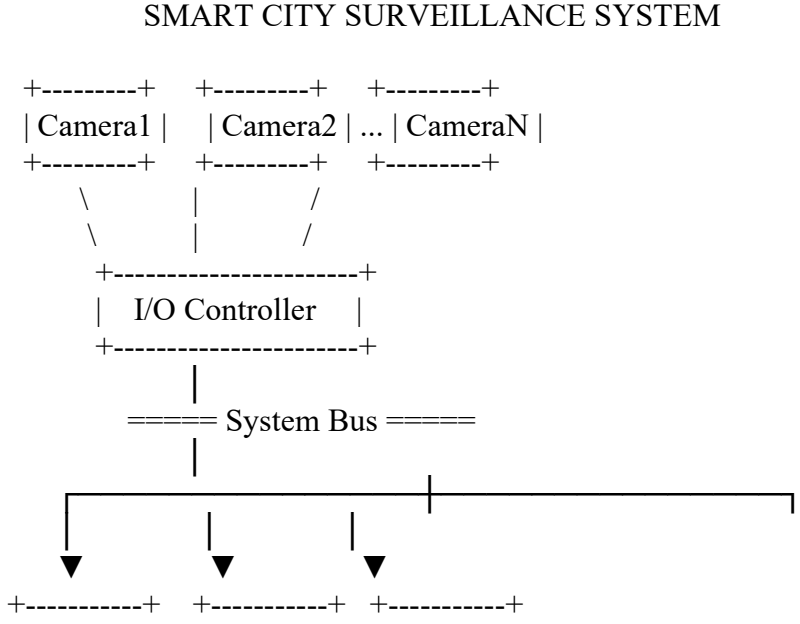
- **Multi-Bus Architecture**

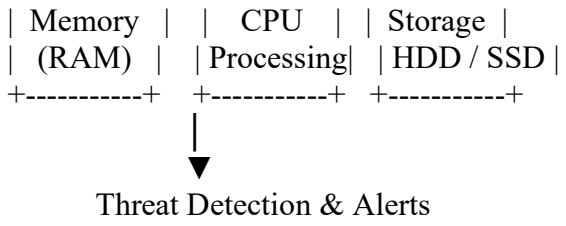
```
        CPU
         |
     High Speed Bus
         |
     Cache Memory
         |
     Memory Bus
         |
        RAM
```

Separate I/O Bus

Camera
SSD
Network
GPU

Main System Architecture Diagram





Conclusion

A smart surveillance system works efficiently only when the CPU, memory, and I/O devices communicate quickly. Using a multi-bus architecture, cache memory, pipelining, and DMA reduces delays and improves threat detection performance.

Question 2: Real-Time Stock Trading System

Introduction

A real-time stock trading system processes millions of instructions every second to buy and sell shares quickly. Speed is very important because even a small delay can affect profits. The processor follows the fetch–decode–execute cycle to execute every instruction. During high workloads, frequent memory access and branch instructions increase the Cycles Per Instruction (CPI), making the processor slower. Using modern processor techniques can reduce execution time and improve system performance.

Applications

- Online stock exchanges
- Banking systems
- Cryptocurrency trading
- Financial analytics
- High-frequency trading

Fetch–Decode–Execute Cycle

The CPU executes every instruction in three main steps.

1. Fetch

The CPU reads the instruction from memory.

2. Decode

The instruction is understood by the control unit.

3. Execute

The CPU performs the required operation and stores the result.

Every instruction follows these steps.

Sample C Program

```
#include <stdio.h>

int main()
{
   int price = 150;
   int target = 120;

   if(price > target)
      printf("SELL\n");
   else
      printf("BUY\n");

   return 0;
}
```

Methods to Reduce CPI

Pipeline

Allows several instructions to execute together.

Cache Memory

Stores frequently used instructions and data.

Branch Prediction

Predicts the next instruction correctly to reduce delays.

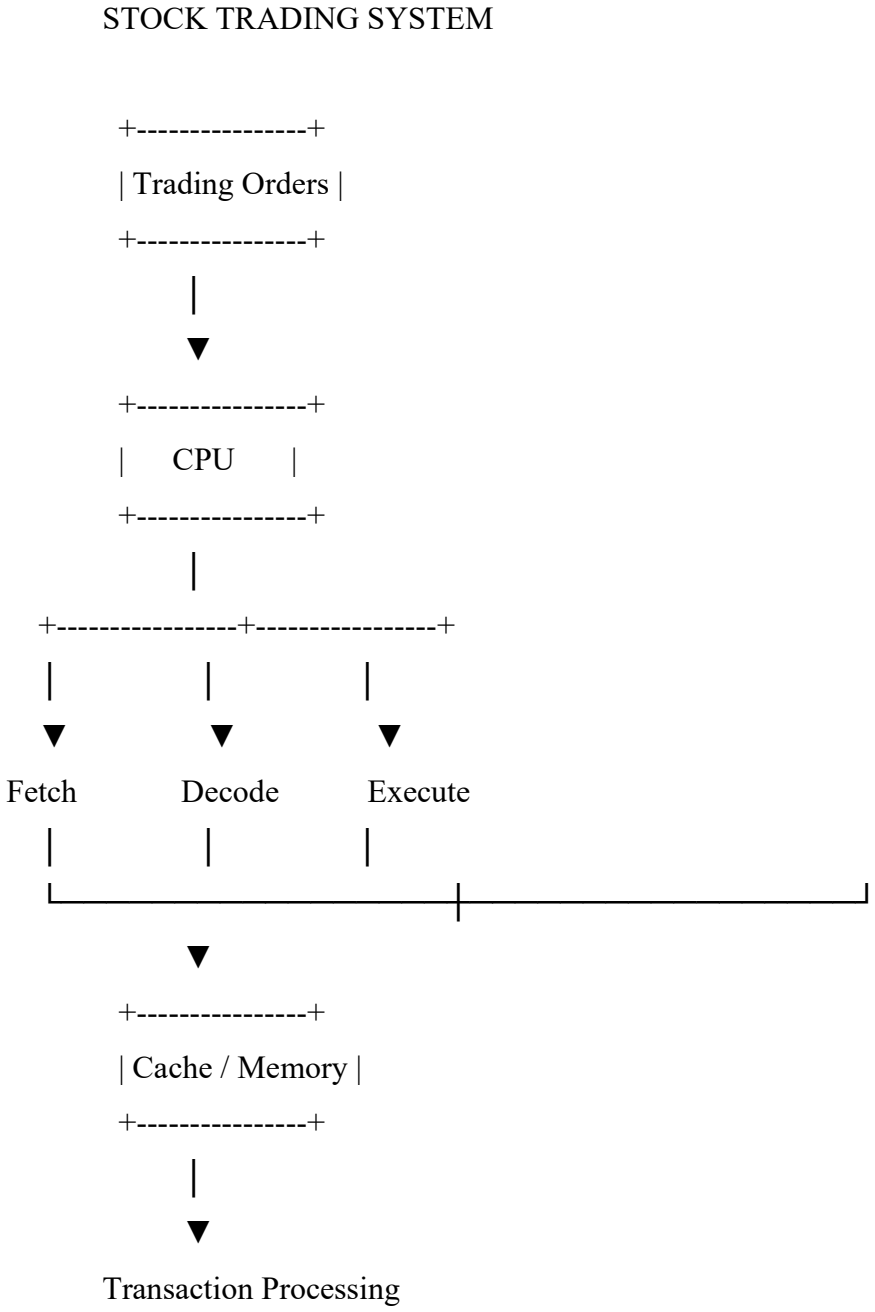
Superscalar Processor

Executes more than one instruction in a single clock cycle.

Multithreading

Allows multiple tasks to run at the same time

Main CPU Execution Diagram



Example

If a processor executes 10 million instructions with a CPI of 4, execution takes longer than a processor with a CPI of 2.

Conclusion

The fetch–decode–execute cycle affects processor speed. High CPI reduces performance, but techniques like pipelining, cache memory, branch prediction, and multithreading improve execution speed and make stock trading systems faster and more reliable.

Question 3: 8085 Temperature Control System

Introduction

The 8085 microprocessor is widely used in embedded systems for industrial automation. In a temperature control system, sensors measure temperature and send data to the microprocessor. The processor reads the data, processes it, and controls devices such as heaters or cooling fans. Correct use of addressing modes is very important because they determine how the processor accesses data. Wrong addressing may lead to incorrect temperature readings and poor system performance.

Applications

- Industrial temperature control
- Boiler systems
- Air conditioning systems
- Food processing
- Medical equipment
- Refrigeration systems

Addressing Modes of 8085

1. Immediate Addressing

The data is directly included in the instruction.

Example:

assembly

```
MVI A,25H
```

2. Register Addressing

Data is transferred between registers.

Example:

assembly

```
MOV A,B
```

3. Direct Addressing

The memory address is given directly.

Example:

assembly

```
LDA 2050H
```

4. Indirect Addressing

The memory address is stored in the HL register pair.

Example:

assembly

```
MOV A,M
```

5. Implied Addressing

The operand is already known.

Example:

assembly

CMA

Causes of Incorrect Readings

- Wrong memory address
- Incorrect HL register value
- Uninitialized registers
- Memory errors
- Sensor timing problems

Solutions

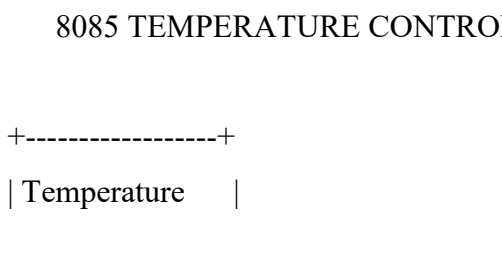
- Initialize registers correctly.
- Use the correct addressing mode.
- Validate sensor values.
- Check memory addresses carefully.
- Test the program before implementation.

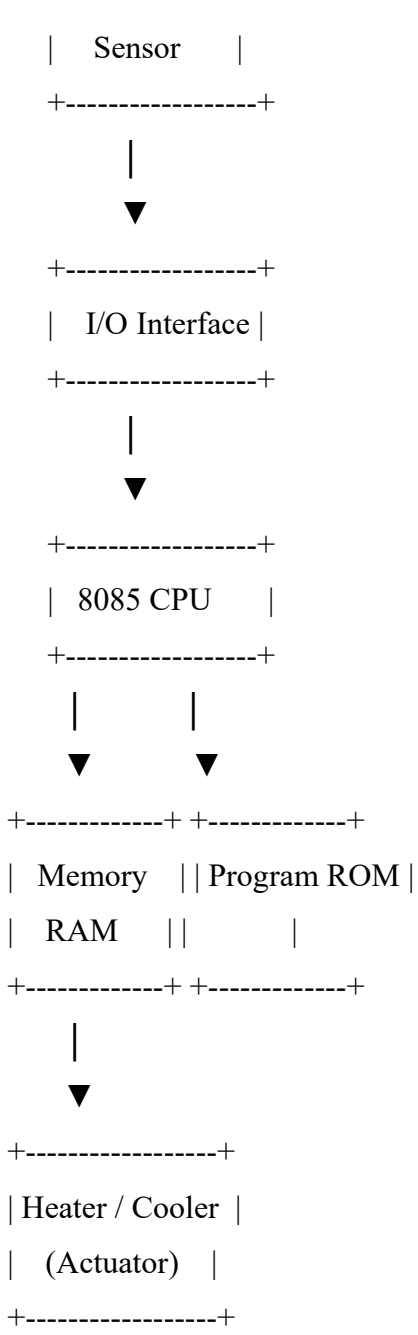
Sample 8085 Program

assembly

```
LDA 2050H

CPI 64H

JC STORE

MVI A,64H

STORE:
STA 3000H
HLT
```

Main Embedded System Diagram





Conclusion

The correct use of 8085 addressing modes ensures accurate data access and reliable temperature control. Proper programming, correct memory addressing, and validation of sensor readings improve the performance and reliability of embedded systems used in industries.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)
Q1					
Q2					
Q3					

Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand